Code Layout, Readability and Reusability

Phase 5: Project Development Phase

1. Purpose of Code Organization

A readable repository helps reviewers understand how the application is built and allows future contributors to make changes without modifying unrelated areas. The project separates the browser interface from the API service and separates AI logic from database setup and request schemas.

2. Frontend Structure

The main frontend application logic is in frontend/src/App.tsx. Styling is maintained in App.css and index.css, while main.tsx starts the React application. The package manifest lists the JavaScript dependencies and scripts. Keeping visual styling separate from the main component improves readability; as the application grows, repeated sections can be extracted into smaller components.

3. Backend Structure

The backend main.py file defines the FastAPI application and API routes. ai_service.py contains the model interaction and response parsing logic. schemas.py defines the request and response data models. supabase_client.py centralizes database client setup, while requirements.txt lists Python dependencies.

4. Separation of Responsibilities

- API routes should coordinate requests rather than contain all prompt logic.
- The AI service should own prompt construction, model calls and response validation.
- Schemas should describe the expected data contract.
- The database client should centralize connection setup.
- The frontend should handle presentation and user actions, while the backend remains responsible for protected service credentials.

5. Naming and Error Handling

Use names that describe the purpose of a function or variable, keep functions focused, and return consistent response structures. Errors should distinguish invalid input, model failures, database failures and missing records. Avoid catching an exception and returning an apparently successful empty itinerary, because that makes debugging difficult.

6. Configuration and Secrets

Environment-specific values should be supplied through private configuration. The source archive should not contain real API keys. A sample environment file may list variable names without values, accompanied by setup instructions. Logs and screenshots should also be checked for accidental secret disclosure.

7. Reusability and Future Refactoring

Potential refactoring includes reusable form controls, a dedicated API client module, shared types for API responses, centralized error messages and automated tests for schema validation. Refactoring should preserve existing behaviour and be performed incrementally, with a working version retained after each change.

8. Review Checklist

- Can a new developer locate the frontend entry point quickly?
- Can the AI prompt and parsing logic be found without searching through all routes?
- Are request and response fields documented?
- Are credentials excluded from source control?
- Are errors specific enough to diagnose?

- Are repeated UI patterns candidates for reusable components?

The source layout described here is based on the supplied project archive. Recommendations are identified as maintenance improvements rather than claims that all practices are already implemented.